

Auto Data Cleaner Pro — Complete Documentation

Version: Enterprise Edition

Language: Python 3.8+

Interface: Tkinter GUI

Last Updated: 2025

Table of Contents

1. [Introduction](#)
 2. [System Requirements](#)
 3. [Installation & Setup](#)
 4. [Application Overview](#)
 5. [UI Reference](#)
 - 5.1 [Header & Status Bar](#)
 - 5.2 [Left Panel](#)
 - 5.3 [Notebook Tabs](#)
 6. [Core Modules](#)
 - 6.1 [EnhancedDataCleaner](#)
 - 6.2 [DataQualityScorer](#)
 - 6.3 [AnomalyDetector](#)
 7. [Cleaning Operations](#)
 8. [Visualization Windows](#)
 9. [Anomaly Detection](#)
 10. [Data Quality Scoring](#)
 11. [Exporting & Saving](#)
 12. [Configuration Reference](#)
 13. [Error Handling](#)
 14. [FAQ & Troubleshooting](#)
 15. [API Reference](#)
-

1. Introduction

Auto Data Cleaner Pro is a desktop application that automates the most common data preparation tasks for analysts, data scientists, and developers. It processes CSV and Excel datasets through a configurable cleaning pipeline, scores data quality, detects anomalies using machine learning, and renders a suite of visualizations — all without writing any code.

Key Capabilities

- Load CSV or Excel files and instantly see quality metrics
- Run a configurable cleaning pipeline in one click
- Score data quality on an A-F grading scale
- Detect anomalous rows using an Isolation Forest model
- Open six different interactive visualization windows
- Export cleaned data as CSV or Excel, and generate a text quality report
- Full operation log with timestamps

2. System Requirements

Requirement	Minimum	Recommended
Python	3.8	3.10+
RAM	4 GB	8 GB+
OS	Windows 10 / macOS 11 / Ubuntu 20.04	Latest
Display	1280×720	1920×1080
Disk	200 MB (deps)	1 GB+ for large datasets

Python Dependencies

Package	Purpose
<code>pandas</code>	DataFrame operations
<code>numpy</code>	Numeric computations
<code>scikit-learn</code>	Isolation Forest, StandardScaler
<code>matplotlib</code>	Charts and plot windows
<code>seaborn</code>	Heatmap rendering
<code>openpyxl</code>	Excel file I/O
<code>chardet</code>	Encoding detection
<code>tkinter</code>	GUI (bundled with Python)

3. Installation & Setup

Step 1 — Clone the Repository

```
bash

git clone https://github.com/your-username/auto-data-cleaner-pro.git
cd auto-data-cleaner-pro
```

Step 2 — Create a Virtual Environment

```
bash

# Create environment
python -m venv venv

# Activate on macOS/Linux
source venv/bin/activate

# Activate on Windows
venv\Scripts\activate
```

Step 3 — Install Dependencies

```
bash

pip install -r requirements.txt
```

requirements.txt:

```
pandas>=1.5.0
numpy>=1.23.0
scikit-learn>=1.1.0
matplotlib>=3.6.0
seaborn>=0.12.0
openpyxl>=3.0.0
chardet>=5.0.0
```

Step 4 — Run

```
bash

python main.py
```

Linux Note — Tkinter

If you see `ModuleNotFoundError: No module named 'tkinter'`:

```
bash

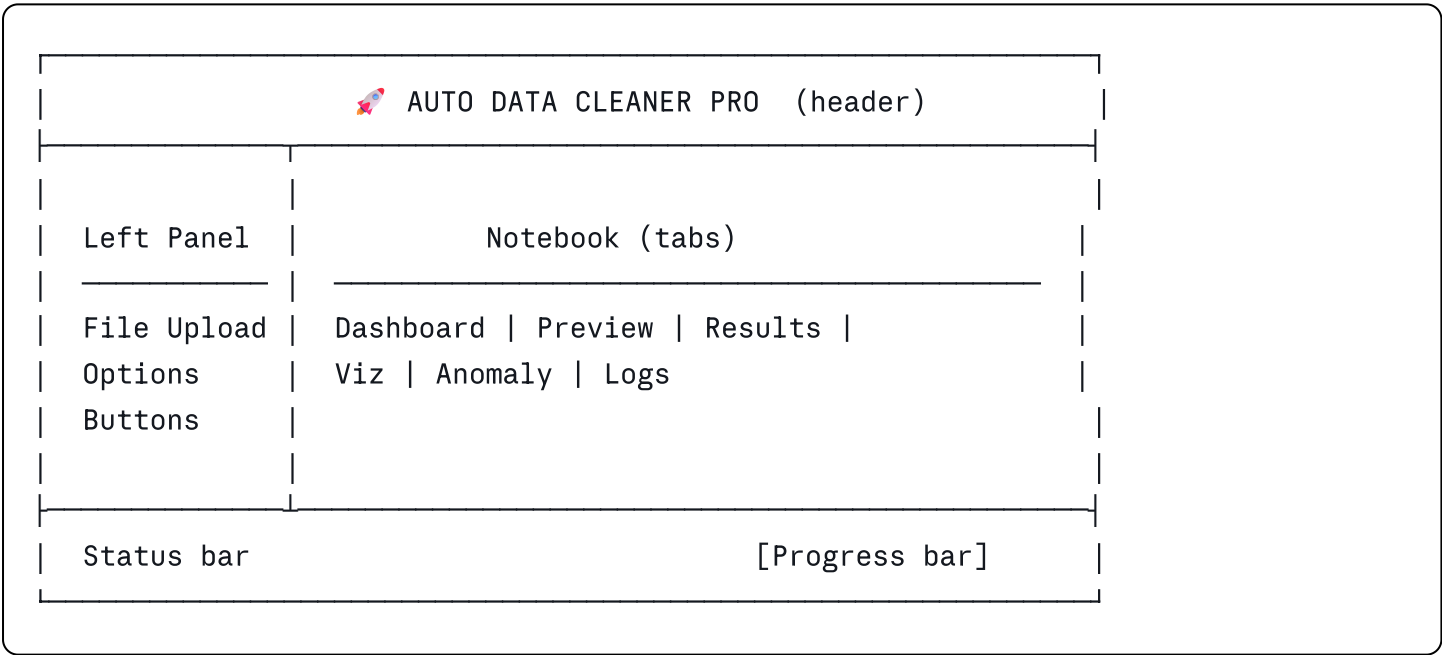
# Ubuntu/Debian
sudo apt install python3-tk

# Fedora
sudo dnf install python3-tkinter

# Arch
sudo pacman -S tk
```

4. Application Overview

The application window has three zones:



5. UI Reference

5.1 Header & Status Bar

The **header** displays the application name. The **status bar** at the bottom shows the current operation state and an animated progress bar that spins during background tasks (file loading, cleaning, anomaly detection).

Status messages:

- ☒ Ready

 — idle, no operation running

-  Loading... — file is being read
-  Cleaning in progress... — cleaning pipeline running
-  Detecting anomalies... — ML model running
-  Loaded N rows, M columns — file loaded successfully
-  Cleaning completed! — pipeline finished

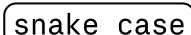
5.2 Left Panel

Data Source

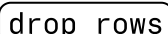
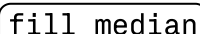
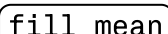
Control	Description
Upload CSV/Excel File	Opens a file dialog filtered to  ,  , 
File Info box	Shows filename, row/column count, size in MB, missing values, and duplicate count after load

Cleaning Options

Five checkboxes, all enabled by default:

Checkbox	What it does
Standardize Column Names	Renames columns to lowercase 
Remove Duplicate Rows	Drops exact duplicate rows
Handle Missing Values	Fills or drops based on strategy
Correct Data Types	Coerces string columns to numeric where possible
Clean Text Data	Strips whitespace from all string columns

Missing Strategy dropdown:

Value	Behavior
	Drops columns >50% missing; fills numeric with median, text with mode
	Drops all rows with any null value
	Fills numeric nulls with column median
	Fills numeric nulls with column mean

Action Buttons

Button	Action
▶ START CLEANING	Runs the cleaning pipeline in a background thread
🔍 DETECT ANOMALIES	Runs Isolation Forest on all numeric columns

💾 Save Data

Button	Action
💾 Save Cleaned Data	Saves cleaned (or current) dataset as CSV or Excel
📄 Export Report	Saves a plain-text quality report

5.3 Notebook Tabs

📊 Dashboard Tab

Displays live metrics after a file is loaded:

Metric	Description
Quality Score	0–100% overall score with A–F grade
Total Rows	Row count
Total Columns	Column count
Missing Values	Total null cell count
Duplicate Rows	Exact duplicate row count
Completeness	% of non-null cells
Uniqueness	Score penalizing duplicate rows
Memory Usage	Dataset RAM footprint (MB)
Data Types	Number of unique dtype categories

The dashboard updates after both initial load and after cleaning (if you choose to replace the original).

Data Preview Tab

Displays the dataset as a scrollable table.

Control	Description
Show dropdown	Toggle between Original Data and Cleaned Data
Rows spinner	Number of rows to display (10–500, default 50)
 Refresh	Reloads the table with current selections

Cells truncated to 50 characters. Null values shown as NULL.

Cleaning Results Tab

A side-by-side comparison showing dataset stats and a list of performed operations:

Before panel shows: rows, columns, missing values, duplicates (original dataset).

After panel shows: same stats post-clean, plus a checklist of operations that ran.

After cleaning completes, a dialog asks whether to replace the working dataset with the cleaned version. Selecting **Yes** updates the Dashboard and Preview tabs.

Visualizations Tab

Six buttons, each opening an independent Tkinter window with a matplotlib canvas and navigation toolbar (zoom, pan, save).

See [Section 8](#) for details on each chart.

Anomaly Detection Tab

Displays the full anomaly detection report after running  **DETECT ANOMALIES**. Shows total rows, numeric column count, anomaly count and percentage, and a list of affected columns.

See [Section 9](#) for the full methodology.

Logs Tab

A scrolling text area with timestamped log entries for every operation:

```
[2025-06-01 14:23:01] [SUCCESS] Load Dataset: Loaded 10,000 rows, 15 columns
[2025-06-01 14:23:04] [SUCCESS] Column Standardization: Renamed 3 columns
[2025-06-01 14:23:04] [SUCCESS] Duplicate Removal: Removed 47 duplicate rows
[2025-06-01 14:23:04] [INFO]     Missing Values: Dropped column 'notes' (72.3%
missing)
[2025-06-01 14:23:04] [SUCCESS] Missing Values: Reduced from 1,240 to 0 missing
values
```

Log levels: `SUCCESS`, `INFO`, `ERROR`.

6. Core Modules

6.1 EnhancedDataCleaner

Handles all file I/O and cleaning logic.

```
python

cleaner = EnhancedDataCleaner(log_callback=my_log_fn)
df = cleaner.load_dataset("data.csv", file_type="csv")
cleaned_df = cleaner.clean_dataset(df, config)
```

Constructor parameters:

Parameter	Type	Description
<code>log_callback</code>	<code>callable</code> or <code>None</code>	Function to receive log strings in real time

Methods:

Method	Parameters	Returns	Description
<code>load_dataset</code>	<code>file_path,</code> <code>file_type</code>	<code>DataFrame</code>	Reads CSV or Excel
<code>standardize_column_names</code>	<code>df</code>	<code>DataFrame</code>	Renames columns to snake_case
<code>remove_duplicates</code>	<code>df</code>	<code>DataFrame</code>	Drops duplicate rows
<code>handle_missing_values</code>	<code>df, strategy,</code> <code>threshold</code>	<code>DataFrame</code>	Fills or drops nulls
<code>correct_data_types</code>	<code>df</code>	<code>DataFrame</code>	Coerces string→numeric
<code>clean_text</code>	<code>df</code>	<code>DataFrame</code>	Strips/normalizes whitespace
<code>clean_dataset</code>	<code>df, config</code>	<code>DataFrame</code>	Runs full configurable pipeline

`clean_dataset` config dictionary:

```
python
config = {
    'standardize_columns': True,
    'remove_duplicates': True,
    'handle_missing': True,
    'correct_types': True,
    'clean_text': True,
    'missing_strategy': 'auto',    # 'auto' | 'drop_rows' | 'fill_median' | 'fill_mean'
    'missing_threshold': 50      # % missing above which a column is dropped (auto)
}
```

After `clean_dataset` runs, the summary is available at `cleaner.cleaning_summary`:

```
python
```

```
{
    'columns_renamed': int,
    'duplicates_removed': int,
    'missing_before': int,
    'missing_after': int,
    'columns_dropped': [str, ...],
    'type_changes': int,
    'cleaning_operations': [str, ...],
    'original_shape': (rows, cols),
    'cleaned_shape': (rows, cols)
}
```

6.2 DataQualityScorer

Calculates a composite quality score from four independent metrics.

```
python

scorer = DataQualityScorer()
result = scorer.calculate_quality_score(df)
```

Return value:

```
python

{
    'total_score': float,          # 0.0-100.0
    'grade': str,                  # 'A' | 'B' | 'C' | 'D' | 'F'
    'description': str,           # Human-readable summary
    'metrics': {
        'completeness': float,
        'uniqueness': float,
        'consistency': float,
        'validity': float
    }
}
```

Grading scale:

Score	Grade	Description
≥ 90	A	Excellent data quality
≥ 75	B	Good data quality
≥ 60	C	Fair data quality
≥ 40	D	Poor data quality
< 40	F	Very poor data quality

6.3 AnomalyDetector

Wraps scikit-learn's `IsolationForest` with automatic scaling.

```
python

detector = AnomalyDetector(contamination=0.1)
results = detector.detect_anomalies(df)
```

Constructor parameters:

Parameter	Type	Default	Description
<code>contamination</code>	<code>float</code>	<code>0.1</code>	Expected fraction of anomalies (0.0-0.5)

Return value:

```
python

{
    'anomalies': [int, ...],          # Row indices flagged as anomalous
    'anomaly_scores': [float, ...],  # Raw isolation scores (lower = more anomalous)
    'anomaly_count': int,
    'anomaly_percentage': float,
    'affected_columns': [str, ...]   # Numeric columns used for detection
}
```

If the dataset has no numeric columns, returns all-zero/empty results rather than raising an error.

7. Cleaning Operations

Column Name Standardization

Transforms column names using these steps in order:

- 1. Strip leading/trailing whitespace
- 2. Convert to lowercase
- 3. Remove all characters except word chars and spaces (`[^\w\s]`)
- 4. Replace whitespace sequences with `_`
- 5. Collapse repeated underscores
- 6. Strip leading/trailing underscores
- 7. Prefix with `col_` if the name starts with a digit
- 8. Fall back to `column` if result is empty
- 9. Append `_N` suffix (N=1,2,3...) to resolve duplicates

Examples:

Original	Standardized
First Name	first_name
Price (USD)	price_usd
123Revenue	col_123revenue
col	col

Duplicate Removal

Uses `pandas.DataFrame.drop_duplicates()` — exact match across all columns. Does not consider near-duplicates or fuzzy matches.

Missing Value Handling

`auto` mode (default):

For each column with missing values:

if missing% > threshold (default 50%):

→ Drop the column entirely

elif column is numeric:

→ Fill with column median

else:

→ Fill with column mode (most frequent value)

If no mode exists: fill with "Unknown"

drop_rows mode:

Calls `df.dropna()` once — removes any row with at least one null.

fill_median / **fill_mean** mode:

Only fills numeric columns. Non-numeric nulls are left as-is.

Data Type Correction

For each `object`-typed column:

- Attempts `pd.to_numeric(..., errors='coerce')`
 - If >70% of values convert successfully, the column is permanently converted to numeric
 - Rows where conversion failed become `NaN`
-

Text Cleaning

For each `object`-typed column:

- Converts all values to string
 - Strips leading/trailing whitespace (`.str.strip()`)
 - Collapses internal whitespace sequences to a single space
-

8. Visualization Windows

Each visualization opens in a separate `Toplevel` window with a matplotlib `FigureCanvasTkAgg` canvas and a `NavigationToolbar2Tk` (zoom, pan, home, save buttons).

Quality Dashboard

Window size: 900×600

Layout: 2×2 subplot grid

Panel	Chart	Data
Top-left	Pie chart	Quality score vs remainder
Top-right	Bar chart	Completeness, Uniqueness, Consistency, Validity
Bottom-left	Horizontal bar	Top 10 columns by missing value count
Bottom-right	Pie chart	Distribution of dtype categories

Missing Values Heatmap

Window size: 800×600

Uses `seaborn.heatmap` on `df.isnull()`. Each cell is white (present) or colored (missing). Y-axis tick labels hidden for large datasets. Colormap: `viridis`.

Correlation Matrix

Window size: 800×700

Requires at least 2 numeric columns.

Uses `matplotlib.imshow` with `coolwarm` colormap. Axis labels rotated 45°. Includes a colorbar.

Data Distribution

Window size: 1000×800

Shows histograms for up to 6 numeric columns, arranged in a 2-column grid. Each histogram uses 30 bins, blue fill, black edge.

Data Types Pie Chart

Window size: 600×500

One pie slice per unique dtype category (e.g., `int64`, `float64`, `object`). Shows percentage labels.

Box Plots

Window size: 1000×600

Side-by-side box plots for up to 6 numeric columns on a single axis. Boxes filled with semi-transparent blue. X-axis labels rotated 45°.

9. Anomaly Detection

Algorithm

Uses **Isolation Forest** — an ensemble method that isolates observations by randomly partitioning features. Anomalous rows are those requiring fewer splits to isolate, indicating they lie far from the majority of the data.

Process

- 1. Select all numeric columns from the dataset
- 2. Fill remaining nulls with each column's median
- 3. Apply `StandardScaler` (zero mean, unit variance)
- 4. Fit `IsolationForest(contamination=0.1, random_state=42)`
- 5. Predict: `-1` = anomaly, `1` = normal
- 6. Compute per-row anomaly scores via `score_samples()`

Interpreting Results

Field	Meaning
<code>anomaly_count</code>	Total rows flagged
<code>anomaly_percentage</code>	% of dataset flagged
<code>anomaly_scores</code>	Lower score = more isolated = more anomalous
<code>affected_columns</code>	Numeric columns used as features

Tuning Contamination

The default `contamination=0.1` assumes ~10% of rows are anomalous. To adjust, modify the `AnomalyDetector` instantiation in `ProfessionalDataCleaner.__init__`:

```
python
self.anomaly_detector = AnomalyDetector(contamination=0.05) # expect ~5% anomalies
```

10. Data Quality Scoring

Metric Definitions

Completeness (weight: 30%)

A dataset with no missing values scores 100%.

Uniqueness (weight: 30%)

```
uniqueness = max(0, 100 - duplicate_row_percentage)
```

A dataset with 10% duplicate rows scores 90%.

Consistency (weight: 20%)

Checks for case variation in text columns. For each object column, if the number of distinct lowercase values is less than the number of distinct original values, a consistency issue is counted.

```
consistency = max(0, 100 - (issues / total_columns × 20))
```

Validity (weight: 20%)

Counts negative values in columns whose name contains `age`, `price`, or `quantity`.

```
invalid_percentage = (negative_values / total_cells) × 100  
validity = max(0, 100 - invalid_percentage × 2)
```

Score Formula

```
total_score = (completeness × 0.30)  
             + (uniqueness × 0.30)  
             + (consistency × 0.20)  
             + (validity × 0.20)
```

11. Exporting & Saving

Save Cleaned Data

After cleaning, click  **Save Cleaned Data**. A file dialog opens:

- `.csv` → `DataFrame.to_csv(path, index=False)`
- `.xlsx` → `DataFrame.to_excel(path, index=False)`

If no cleaning has been run, the original loaded dataset is saved.

Export Quality Report

Click  **Export Report** to save a `.txt` file containing:

```
AUTO DATA CLEANER PRO - QUALITY REPORT
=====

Generated: 2025-06-01 14:23:01
File: sales_data.csv

DATASET SUMMARY
-----
Rows: 9,953
Columns: 14
Missing Values: 0
Duplicate Rows: 0

QUALITY SCORE
-----
Overall: 91.45%
Grade: A
Description: Excellent data quality

METRICS
-----
Completeness: 100.0%
Uniqueness: 99.5%
Consistency: 95.0%
Validity: 100.0%
```

12. Configuration Reference

Full Config Dictionary

```
python
```

```

config = {
    # Cleaning toggles
    'standardize_columns': bool,    # Default: True
    'remove_duplicates': bool,    # Default: True
    'handle_missing': bool,    # Default: True
    'correct_types': bool,    # Default: True
    'clean_text': bool,    # Default: True

    # Missing value strategy
    'missing_strategy': str,    # 'auto' | 'drop_rows' | 'fill_median' | 'fill_me
    'missing_threshold': int,    # 0-100, column drop threshold in auto mode (defa
}

```

Color Theme

All UI colors are defined in `ProfessionalDataCleaner.colors`:

```

python

colors = {
    'primary': '#0f3460',    # Dark blue – header, buttons
    'secondary': '#16213e',    # Darker blue – main background
    'accent': '#e94560',    # Red-pink – upload button, titles
    'success': '#4ecdc4',    # Teal – clean button, positive metrics
    'warning': '#ff6b6b',    # Coral – anomaly button, negative metrics
    'text': '#eeeeee',    # Off-white – primary text
    'text_light': '#cccccc',    # Light grey – secondary text
    'card': '#1a1a2e',    # Card background
    'card_dark': '#0f0f1a'    # Text area background
}

```

13. Error Handling

The application handles errors at two levels:

Background Thread Errors

File loading, cleaning, and anomaly detection run in `threading.Thread`. Errors are caught and routed back to the main thread via `root.after(0, lambda: ...)`:

```

python

```

```
except Exception as e:
    self.root.after(0, lambda: messagebox.showerror("Error", str(e)))
    self.root.after(0, lambda: self.update_status("Ready", False))
```

This prevents GUI freezes. Error details appear in a modal dialog.

Logged Errors

Errors during cleaning steps are logged with level `ERROR` and written to the Logs tab:

```
[2025-06-01 14:23:01] [ERROR] Load Dataset: Failed: [Errno 2] No such file or
directory
```

Common Errors

Error	Cause	Fix
No module named 'sklearn'	scikit-learn not installed	<code>pip install scikit-learn</code>
No module named 'tkinter'	tkinter missing on Linux	<code>sudo apt install python3-tk</code>
Engine 'openpyxl' not found	openpyxl not installed	<code>pip install openpyxl</code>
Insufficient Data (correlation)	Fewer than 2 numeric columns	Use a dataset with numeric data
Visualization window blank	Dataset not loaded	Upload a file first

14. FAQ & Troubleshooting

Q: The app window is too small / cuts off.

A: The default geometry is `1600x900`. On smaller screens, resize the window manually or reduce font scaling in your OS.

Q: Cleaning ran but my data looks the same.

A: Check the Logs tab. It may be that no issues were found (no duplicates, no missing values). Also confirm you clicked **Yes** when prompted to replace the original.

Q: Can I undo cleaning?

A: The original dataset is preserved internally as `original_dataset`. If you clicked **No** when asked to replace, the working dataset is unchanged. There is no multi-step undo — save your original file before cleaning if you need a rollback.

Q: Anomaly detection always shows 10% of rows as anomalous.

A: That's expected — the default `(contamination=0.1)` parameter tells the model to expect ~10% anomalies. Lower it if your data is cleaner. See [Section 9](#).

Q: Heatmap is very slow on large datasets.

A: seaborn heatmaps scale poorly above ~500 columns. Consider sampling the dataset before opening the heatmap window.

Q: Excel files fail to load.

A: Ensure `(openpyxl)` is installed (`(pip install openpyxl)`). `(.xls)` files (old Excel format) are not supported — convert to `(.xlsx)` first.

Q: How do I add a new cleaning operation?

A: Add a method to `(EnhancedDataCleaner)`, add a key to the `(config)` dict, and wire it up inside `(clean_dataset)`. Add a corresponding checkbox in `(setup_left_panel)`.

15. API Reference

EnhancedDataCleaner

python

```
class EnhancedDataCleaner:
    def __init__(self, log_callback=None): ...
    def log(self, operation: str, details: str, level: str = 'INFO') -> None: ...
    def load_dataset(self, file_path: str, file_type: str = 'csv') -> pd.DataFrame:
    def standardize_column_names(self, df: pd.DataFrame) -> pd.DataFrame: ...
    def remove_duplicates(self, df: pd.DataFrame) -> pd.DataFrame: ...
    def handle_missing_values(self, df: pd.DataFrame, strategy: str = 'auto', thresh
    def correct_data_types(self, df: pd.DataFrame) -> pd.DataFrame: ...
    def clean_text(self, df: pd.DataFrame) -> pd.DataFrame: ...
    def clean_dataset(self, df: pd.DataFrame, config: dict) -> pd.DataFrame: ...
```

DataQualityScorer

python

```
class DataQualityScorer:
    def calculate_quality_score(self, df: pd.DataFrame) -> dict: ...
    def _completeness_score(self, df: pd.DataFrame) -> float: ...
    def _uniqueness_score(self, df: pd.DataFrame) -> float: ...
    def _consistency_score(self, df: pd.DataFrame) -> float: ...
    def _validity_score(self, df: pd.DataFrame) -> float: ...
```

AnomalyDetector

python

```
class AnomalyDetector:
    def __init__(self, contamination: float = 0.1): ...
    def detect_anomalies(self, df: pd.DataFrame) -> dict: ...
```

ProfessionalDataCleaner

python

```
class ProfessionalDataCleaner:
    def __init__(self): ...
    def setup_ui(self): ...
    def setup_left_panel(self, parent): ...
    def setup_notebook(self, parent): ...
    def upload_file(self): ...
    def start_cleaning(self): ...
    def detect_anomalies(self): ...
    def save_cleaned_data(self): ...
    def export_report(self): ...
    def open_dashboard_window(self): ...
    def open_heatmap_window(self): ...
    def open_correlation_window(self): ...
    def open_distribution_window(self): ...
    def open_piechart_window(self): ...
    def open_boxplot_window(self): ...
    def run(self): ...
```